Team number: 47

M1: Approach — Financial literacy through an interactive Indian town

1. Problem statement and understanding

The allocated financial-literacy problem asks for accessible, game-based learning that helps young people make informed financial decisions and supports an NGO learning context. We understand the core gap as the difference between recognizing financial terms and applying them safely: opening an eligible bank account, checking a digital payment, spotting fraud, budgeting, and comparing borrowing costs. A quiz on definitions alone does not demonstrate that a learner can handle a new situation.

Our initial audience is Grade 11–12 learners in Hyderabad, with English and Telugu support and a mobile-first experience. The existing project brief identifies a potential NGO audience of approximately 1,500 learners; this is a planning context, not a verified deployment or study sample. Short sessions, interrupted connectivity, shared devices, and different reading speeds shape the design. We measure financial reasoning and improvement, not virtual wealth or possessions.

The planned curriculum covers four pillars: plan and budget; save and manage; understand investing; stay financially safe. Investment content explains risk and trade-offs without recommending securities or promising returns. All identities, documents, balances, payments, and borrowing cases are fictional; real financial identifiers and credentials are never required.

2. Idea / abstract

Learners explore an illustrated Indian town and visit familiar institutions: a bank, market, phone shop, and lending desk. Each mission presents a short concept, asks the learner to act in a simulated interface, shows a clear consequence, and checks understanding with a changed scenario. For example, the learner inspects fictional documents at the bank rather than memorizing account-opening terminology; a later loan comparison distinguishes a low monthly payment from total repayment and affordability.

The loop is: learn a concept → travel to a location → make a decision → see its consequence → receive explained feedback → unlock a skill → apply it again. Attempts earn 0–3 stars from an explicit rubric. Mistakes produce a constructive review and a retry; first-attempt and best-attempt results remain separate. Reading speed never changes the score. Critical unresolved safety errors prevent success, while replay enables recovery. Completed learning changes the town, such as a teller recognizing the returning player, making progress visible across sessions.

3. High-level approach and delivery scope

We will first prove one complete journey: travel to the bank, inspect fictional documents, complete a banking challenge, make and understand a mistake, retry, unlock a skill, and reload with progress intact. Only after that gate passes will we extend the demo to UPI, fraud detection, and loan-cost comparison, including an explicit negative borrowing outcome. The broader twelve-level curriculum, server synchronization, educator dashboard, and optional AI dialogue are later work, not promised hackathon completion.

The proposed architecture separates presentation, lessons, and deterministic rules. **Input → town / lesson interface → interaction events → validation, finance and scoring → saved attempt history → derived stars, skills and world changes.** React provides accessible HTML panels and text; PixiJS renders the town; one GSAP timeline coordinates movement. Keyboard and button alternatives accompany gestures, and reduced motion changes presentation without changing lesson outcomes.

Team number: 47

3. High-level approach (continued)

Curated lesson definitions use reusable interaction components, starting with document inspection and fictional forms, then payment checks, message triage, and offer comparison. TypeScript and Zod validate lesson data, events, and saves. A pure core calculates money in integer paise and derives scores and world state from recorded attempts, keeping the same rules in play and tests. Financial claims will be linked to authoritative Indian sources and educator-reviewed before a lesson ships.

Zustand connects state to the interface; Dexie/IndexedDB stores fictional local profiles and progress. A service worker will cache the shell and lesson assets for play after the first successful load. Save validation, migrations, and recovery snapshots protect interrupted sessions and corrupt saves. English/Telugu text stays in the HTML interface for accessibility and proper script rendering. The guided experience needs neither a backend nor an LLM; optional external services cannot block learning.

Privacy is enforced by minimizing data: no real names, contact details, Aadhaar/PAN, bank/UPI identifiers, PINs, or OTPs; no analytics, ads, or external model calls during core play. Local profile separation and deletion controls address ordinary shared-device use; local profiles are not authentication against someone with device access. Any future remote student records need a separate consent and authorization design before an NGO pilot. Sponsor branding remains subject to permission, with a neutral educational centre as the default.

4. Planned testing methodology

Rules and calculations. Use Vitest and property-based tests for finance, scoring, state derivation, and save recovery. Check integer-paise calculations against reference values, including zero-interest loans; for the same positive rate and principal, longer tenure should reduce EMI while increasing total interest. Test critical safety errors, hint deductions, transfer-question mistakes, and preservation of the first attempt when best stars improve.

Interactions and continuity. Component tests exercise correct and incorrect choices and their emitted events. Playwright will run at 360×740 and 412×915 touch viewports and 1440×900 desktop. The acceptance journey must show one gesture moving exactly one stop, bank completion, a scripted 2-star attempt, a 3-star retry, a lit skill node, a changed teller, and identical state after reload. Test panel scrolling without town movement, interrupted saves, offline play after caching, corrupt-save recovery, profile switching, and deletion.

Accessibility, content, and performance. Run automated accessibility checks plus keyboard-only and reduced-motion journeys; review Telugu and expanded-text screenshots for clipping. Content validation requires source references, review stamps, and translation coverage. Asset checks cover consistency and licensing. Target initial JavaScript <=250 KB gzipped, first-level assets <=1.5 MB, mobile LCP <=2.5 seconds under simulated 4G, and travel >=45 fps under CPU throttling. Confirm gesture feel and readability on a real Android phone; automated checks do not replace that test.

Privacy and educational value. Capture network traffic and fail unapproved outbound requests; test that normal profile switching does not mix progress. Review forms, logs, error screens, and any exports for accidental identifiers. An educator will review banking, fraud, and borrowing claims and Telugu wording. With appropriate approvals and consent, a later small learner pilot will compare first-attempt and transfer-scenario accuracy, retry improvement, completion, and ability to explain a safer choice. These are proposed measures, not results or evidence of effectiveness.

Evidence and release gate. CI will run type checks, lint, tests, build, and content/asset gates; end-to-end and performance checks will provide reproducible evidence. We will prioritize a correct, private, accessible first journey over additional incomplete levels. M2 will report actual experiments, measured results, test progress, and remaining work. This M1 describes the planned approach; it does not claim completed gameplay, learning gains, or deployment readiness.